



Nombre:

Grupo:

Pregunta 1 (2 puntos):

Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

- [] En un AVL puede haber un nodo hoja a profundidad 5, mientras otro se encuentra a profundidad 8.
- [] Para eliminar la recursividad en el recorrido en anchura de un árbol hace falta utilizar una pila.
- [] En una lista simplemente enlazada, eliminar el elemento que ocupa la cola requiere O(n) accesos.
- [] El fichero de índice sobre un archivo siempre contiene la misma información que el índice en memoria.
- [] En una malla regular, acceder a la lista de elementos que están en una casilla es O(1).
- [] La implementación de una cola con prioridad es igual de eficiente hacerlo mediante vectores que con listas simplemente enlazadas.
- [] Obtener el valor de menor prioridad en un heap se puede realizar en tiempo constante O(1).
- [] La dispersión doble ayuda a eliminar tanto los agrupamientos primarios como secundarios.
- [] La implementación de matrices dispersas mediante listas comparado con vectores bidimensionales implica un ahorro de espacio pero es menos eficiente en los accesos.
- [] En un quadtree, localizar el elemento más cercano a otro, como mucho implica acceder a los nodos hermanos del nodo donde se encuentra el elemento.

Pregunta 2 (2 puntos):

Generar paso a paso el árbol B de orden 3 tras la inserción (I) y borrado (B) de los siguientes datos correspondientes a cuentas de usuario de un sistema web: I(lola10), I(juanfornite), I(josephe), I(kiko20), I(terezi), I(lula17), I(sancholo), I(dani02), I(albaluna), B(kiko20), B(dani02), I(juansinganas).

Pregunta 3 (3 puntos):

- (a) Implementad la primitiva `vector<T> orden_inverso(AVL a)` que devuelve un vector con los elementos almacenados en un vector en orden inverso (suponiendo que en el AVL se ordenan de menor a mayor, en el vector deben estar ordenados de mayor a menor).
- (b) Dada una malla regular, implementad la primitiva `float factorDispersion()` que devuelve un valor en [0,1] indicando el porcentaje de celdas vacías respecto al número total de celdas ($valor = \frac{n^0 \text{ celdas vacías}}{n^0 \text{ celdas totales}}$).

Pregunta 4 (3 puntos):

Una cadena de gasolineras de bajo coste está totalmente automatizada de manera que sólo sirve a clientes dados de alta en el sistema, sin necesidad de contratar personal. Cada cliente se da de alta en una aplicación móvil y cuando se acerca a una gasolinera solicita el suministro. Cada gasolinera es capaz de gestionar los niveles de sus depósitos de forma automática y llamar a la central para que le suministre si baja de un nivel. Las gasolineras tienen cinco depósitos cada una, correspondientes a tres tipos de diésel y dos de gasolina. Cada depósito tiene un nombre identificativo, por ejemplo: gasolina95.

Cliente: Cuando un cliente se acerca a una gasolinera, solicita repostar con `Cliente::solicitarRepostaje()` llamando a `CentralGasol::solicitarRepostaje()`. Para que pueda repostar se debe cumplir que esté a menos de 100 metros de una gasolinera (la gasolinera se devuelve por la función), que esté dado de alta como cliente en la central (CentralGasol) y que además haya al menos 1000 litros de combustible en esa gasolinera. Una vez hecha la solicitud, ya se puede llamar a `Cliente::echarCarburante()`, que le devuelve el coste del depósito que ha echado. Esta función debe llamar a `GasolineraAut::repostarVehiculo()`. El cliente va acumulando puntos cada vez que reposta y luego podrá convertir en combustible (se explica más adelante).

CentralGasol: Representa a la compañía de gasolineras. Tiene camiones que van siempre llenos con la misma carga de combustible (`litrosCamion`). La función `CentralGasol::solicitarRepostaje()` comprueba que el dni del cliente está en el sistema, y devuelve la gasolinera que está más cercana, siempre que se localice a menos de 100 metros de ésta. La función UTM está ya implementada junto con la función distancia `UTM::distancia()`. Tampoco se pide acelerar este proceso de búsqueda mediante una estructura de datos espacial. Cuando alguna gasolinera se va quedando sin combustible (menos de 5000 litros), la gasolinera enviará a la central una llamada de alarma con `GasolineraAut::avisoCentral()`, llamando a `CentralGasol::alarmaSuministro()`. Esta función debe dar la orden de llevar combustible esa gasolinera con `CentralGasol::enviarSuministro()`. Esta función debe llamar a `GasolineraAut::rellenarDepositoGasolinera()`.

GasolineraAut: Representa a cada una de las gasolineras de la compañía. Tiene un id y una coordenada UTM que sirve para determinar su posición. Si alguno de los 5 depósitos de combustible se queda por debajo de 5000 litros, debe realizar una llamada a la central con `GasolineraAut::alarmaSuministro()`. La función `GasolineraAut::rellenarDepositoGasolinera()` llama a `Deposito::rellenarDeposito()` del depósito correspondiente, que lo llena con los litros que caben en un camión. La función `GasolineraAut::repostarVehiculo()` llama a `Deposito::repostarVehiculo()` para repostarle con el combustible elegido.

Deposito: Es cada uno de los 5 depósitos de combustible de una gasolinera y se identifica por un nombre del tipo: Gasolina95, etc. Tiene una serie de litros en stock que se van decrementando en cada repostaje. Cuando se va quedando vacío, se rellena con los litros que entran en un camión con `Deposito::rellenarDeposito()`. Cada depósito identifica todos los repostajes que realiza con `Deposito::repostarVehiculo()`.

RepostajeVehiculo: representa cada vez que llenamos el depósito, y se enlaza con cliente que está echando combustible. Cada repostaje corresponde a unos litros concretos. La función `RepostajeVehiculo::realizarRepostaje()` determina el precio del repostaje y actualiza los puntos del cliente. Por cada 15 litros repostados en cada repostaje el cliente acumula 1 punto. Por cada punto acumulado el cliente puede repostar un litro gratis. Cuando el cliente llena un número de litros menor que los puntos acumulados, el repostaje le sale gratis y en ese caso la función `Cliente::echarCorburante()` devuelve 0.

